

第7章 DQN 算法

DQN 算法用来解决连续状态下离散动作的问题

7.2 Cart Pole 环境



车杆，它的状态值就是连续，动作值离散

在车杆环境中，有一辆小车，智能体的任务是通过左右移动保持车上的杆竖直，若杆的倾斜度过大，或者车子离初始位置左右的偏离程度过大，或者坚持时间达到 200 帧，则游戏结束。

CartPole 环境的状态空间

维度	意义	最小值	最大值
0	车的位置	-2.4	2.4
1	车的速度	-Inf	Inf
2	杆的角度	$\sim -41.8^\circ$	$\sim 41.8^\circ$
3	杆尖端的速度	-Inf	Inf

CartPole 环境的动作空间

标号	动作
0	向左移动小车
1	向右移动小车

7.3 DQN

类似车杆的环境中得到动作价值函数 $Q(s, a)$ ，由于状态每一维度都是连续的，无法用表格记录。通过函数拟合思想解决。

若动作是连续的，神经网络的输入是状态 s 和动作 a ，然后输出一个标量，表示状态 s 下采取动作 a 能获得的价值。通过神经网络来拟合。

假设神经网络用来拟合函数 w 的参数是，即每一个状态 s 下所有可能动作 a 的 Q 值， $Q_w(s, a)$ 。将用于拟合函数 Q 函数的神经网络称为 Q 网络。

回顾 Q -learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a' \in A} Q(s', a') - Q(s, a)]$$

用时序差分学习目标 $r + \gamma \max_{a' \in A} Q(s', a')$ 来增量式更新 $Q(s, a)$ 。

要使 $Q(s, a)$ 和 TD 目标 $r + \gamma \max_{a' \in A} Q(s', a')$ 靠近，于是，对于一组数据 $\{(s_i, a_i, r_i, s'_i)\}$ ，我们可以自然地将 Q 网络的损失函数构造为均方误差形式：

$$w^* = \arg \min_w \frac{1}{2N} \sum_{i=1}^N [Q_w(s_i, a_i) - (r_i + \gamma \max_{a'} Q_w(s_i, a'))]^2$$

7.3.1 经验回放 (experience replay)

为了更好地将 Q-learning 和深度神经网络结合, DQN 算法采用经验回放方法.

维护一个 回放缓冲区, 将每次环境中采样得到的四元组数据 (s, a, r, s') 存到回放缓冲区中, 训练 Q 网络的时候再从回放缓冲区中随机采样若干数据进行训练.

两个作用.

- (1) 使样本满足独立假设 (2) 提高样本效率

7.3.2 目标网络 (target network)

DQN 算法目标是让 $Q_w(s, a)$ 逼近 $r + \max_{a'} Q_w(s', a')$.

为了解决 TD 误差本身包含神经网络输出, 在更新网络参数的同时目标不断改变易造成网络训练不稳定的问题. DQN 使用目标网络思想.

用到两套 Q 网络.

- (1) 原来的训练网络 $Q_w(s, a)$ 用来计算原系的损失函数:

$$\frac{1}{2} [Q_w(s, a) - (r + \max_{a'} Q_{w^-}(s', a'))]^2 \text{ 中的 } Q_w(s, a)$$

- (2) 目标网络 $Q_{w^-}(s, a)$, 用于计算原损失函数中的:

$$(r + \max_{a'} Q_{w^-}(s', a')) \text{ 项.}$$

~~其中~~ 其中 w^- 表示目标网络中的参数.

目标网络使用训练网络的一套较旧的参数, 训练网络使用 $Q_w(s, a)$ 在训练中每一步都更新, 而目标网络的参数每隔 C 步才会与训练网络同步一次. **让目标网络相对于训练网络更加稳定.**

DQN 具体流程:

用随机的网络参数 w 初始化网络 $Q_w(s, a)$

复制相同的参数 $w^- \leftarrow w$ 初始化目标网络 Q_{w^-}

初始化经验回放池 R

for 序列 $e = 1 \rightarrow E$ do:

 获取环境初始状态 s_1

 for 时间步 $t \rightarrow T$ do:

 根据当前网络 $Q_w(s, a)$ 以 ϵ -greedy 策略选择动作 a_t

 执行动作 a_t , 获得回报 r_t , 环境状态变为 s_{t+1}

 将 (s_t, a_t, r_t, s_{t+1}) 存进回放池 R 中

 若 R 中数据足够, 从 R 中采样 N 个数据 $\{(s_i, a_i, r_i, s_{i+1})\}_{i=1}^N$

 对每个数据, 用目标网络计算 $y_i = r_i + \gamma \max_a Q_{w^-}(s_{i+1}, a)$

 最小化目标损失 $L = \frac{1}{N} \sum_i (y_i - Q_w(s_i, a_i))^2$, 以此更新 Q_w

 更新目标网络

 end for

end for

9. 策略梯度算法

9.2 策略梯度

假设目标策略 π_θ 是一个随机性策略, 并且处处可微, θ 是对应的参数。

目标函数:

$$J(\theta) = E_{s_0} [V^{\pi_\theta}(s_0)]$$

其中, s_0 表示初始状态。对目标函数求导(对 θ), 得到导数后用梯度上升方法来最大化该目标。

推导:

$$\begin{aligned} \nabla_\theta J(\theta) &\propto \sum_{s \in S} V^{\pi_\theta}(s) \sum_{a \in A} (s, a) \nabla_\theta \pi_\theta(a|s) \\ &= \sum_{s \in S} V^{\pi_\theta}(s) \sum_{a \in A} (a|s) Q^{\pi_\theta}(s, a) \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)} \\ &= E_{\pi_\theta} [Q^{\pi_\theta}(s, a) \nabla_\theta \log \pi_\theta(a|s)] \end{aligned}$$

在计算策略梯度的公式中, 要用到 $Q^{\pi_\theta}(s, a)$ 。可用多种方法对它进行估计。REINFORCE 算法采用蒙特卡洛方法来估计 $Q^{\pi_\theta}(s, a)$ 其策略梯度为:

$$\nabla_\theta J(\theta) = E_{\pi_\theta} \left[\sum_{t=0}^T \left(\sum_{t'=t}^T \gamma^{t'-t} r_{t'} \right) \nabla_\theta \log \pi_\theta(a_t | s_t) \right]$$

策略梯度证明

Date.

$$\text{证: } \nabla_{\theta} J(\theta) \propto \sum_{s \in S} V^{\pi_{\theta}}(s) \sum_{a \in A} Q^{\pi_{\theta}}(s, a) \nabla_{\theta} \pi_{\theta}(a|s)$$

推导状态价值函数:

$$\nabla_{\theta} V^{\pi_{\theta}}(s) = \nabla_{\theta} \left(\sum_{a \in A} \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a) \right)$$

$$= \sum_{a \in A} (\nabla_{\theta} \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a) + \pi_{\theta}(a|s) \nabla_{\theta} Q^{\pi_{\theta}}(s, a))$$

$$= \sum_{a \in A} (\nabla_{\theta} \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a) + \pi_{\theta}(a|s) \nabla_{\theta} \sum_{s', r} P(s', r|s, a) (r + \gamma V^{\pi_{\theta}}(s')))$$

$$= \sum_{a \in A} (\nabla_{\theta} \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a) + \gamma \pi_{\theta}(a|s) \sum_{s', r} P(s', r|s, a) \nabla_{\theta} V^{\pi_{\theta}}(s'))$$

$$= \sum_{a \in A} (\nabla_{\theta} \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a) + \gamma \pi_{\theta}(a|s) \sum_{s'} P(s'|s, a) \nabla_{\theta} V^{\pi_{\theta}}(s'))$$

简化表示, 让 $\phi(s) = \sum_{a \in A} \nabla_{\theta} \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a)$, 定义 $d^{\pi_{\theta}}(s \rightarrow x, k)$ 为策略 π 从状态 s 出发后到达状态 x 的概率。

$$\nabla_{\theta} V^{\pi_{\theta}}(s) = \phi(s) + \gamma \sum_a \pi_{\theta}(a|s) \sum_{s'} P(s'|s, a) \nabla_{\theta} V^{\pi_{\theta}}(s')$$

$$= \phi(s) + \gamma \sum_a \sum_{s'} \pi_{\theta}(a|s) P(s'|s, a) \nabla_{\theta} V^{\pi_{\theta}}(s')$$

$$= \phi(s) + \gamma \sum_{s'} d^{\pi_{\theta}}(s \rightarrow s', 1) \nabla_{\theta} V^{\pi_{\theta}}(s')$$

$$= \phi(s) + \gamma \sum_{s'} d^{\pi_{\theta}}(s \rightarrow s', 1) [\phi(s') + \gamma \sum_{s''} d^{\pi_{\theta}}(s' \rightarrow s'', 1) \nabla_{\theta} V^{\pi_{\theta}}(s'')]$$

$$= \phi(s) + \gamma \sum_{s'} d^{\pi_{\theta}}(s \rightarrow s', 1) \phi(s') + \gamma^2 \sum_{s''} d^{\pi_{\theta}}(s \rightarrow s'', 2) \nabla_{\theta} V^{\pi_{\theta}}(s'')$$

$$= \phi(s) + \gamma \sum_{s'} d^{\pi_{\theta}}(s \rightarrow s', 1) \phi(s') + \gamma^2 \sum_{s''} d^{\pi_{\theta}}(s \rightarrow s'', 2) \phi(s'') + \gamma^3 \sum_{s'''} d^{\pi_{\theta}}(s \rightarrow s''', 3) \nabla_{\theta} V^{\pi_{\theta}}(s''')$$

$$= \dots$$

$$= \sum_{x \in S} \sum_{k=0}^{\infty} \gamma^k d^{\pi_{\theta}}(s \rightarrow x, k) \phi(x)$$

No.

Date.

定义 $\eta(s) = E_{s_0} \left[\sum_{k=0}^{\infty} \gamma^k d^{\pi_0}(s_0 \rightarrow s, k) \right]$ 。至此，目标函数：

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} E_{s_0} [V^{\pi_{\theta}}(s_0)]$$

$$= \sum_s E_{s_0} \left[\sum_{k=0}^{\infty} \gamma^k d^{\pi_{\theta}}(s_0 \rightarrow s, k) \right] \phi(s)$$

$$= \sum_s \eta(s) \phi(s)$$

$$= \left(\sum_s \eta(s) \right) \sum_s \frac{\eta(s)}{\sum_s \eta(s)} \phi(s)$$

$$\propto \sum_s \frac{\eta(s)}{\sum_s \eta(s)} \phi(s)$$

$$= \sum_s V^{\pi_0}(s) \sum_a Q^{\pi_0}(s, a) \nabla_{\theta} \pi_{\theta}(a|s)$$

证明完毕。

10. Actor-Critic 算法

Actor-Critic 算法本质上是基于策略梯度算法

在策略梯度中, 可把梯度写为:

$$J = E \left[\sum_{t=0}^T \psi_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

其中, ψ_t 可有多种形式:

1. $\sum_{t'=0}^T \gamma^{t'} r_{t'}$: 轨迹回报

2. $\sum_{t'=t}^T \gamma^{t'-t} r_{t'}$: 动作 a_t 之后的回报

3. $\sum_{t'=t}^T \gamma^{t'-t} r_{t'} - b(s_t)$: 基准线版本改进

4. $Q^{\pi_{\theta}}(s_t, a_t)$: 动作价值函数

5. $A^{\pi_{\theta}}(s_t, a_t)$: 优势函数

6. $r_t + \gamma V^{\pi_{\theta}}(s_{t+1}) - V^{\pi_{\theta}}(s_t)$: 时序差分残差

Actor 用策略梯度更新, Critic 用梯度下降方法更新. 将 Critic 价值网络表示为 V_w , 参数为 w . 于是, 可采取时序差分残差方式, 对单个数据定义如下价值函数的损失函数:

$$L(w) = \frac{1}{2} (r_t + \gamma V_w(s_{t+1}) - V_w(s_t))^2$$

No.

Date.

将上式中 $r + \gamma V_w(s_{t+1})$ 作为时序差分目标, 不会产生梯度来更新价值函数. 价值梯度为:

$$\nabla_w L(w) = -(r + \gamma V_w(s_{t+1}) - V_w(s_t)) \nabla_w V_w(s_t).$$

Actor-Critic 算法具体流程:

· 初始化策略网络参数 θ , 价值网络 w

· for 序列 $e \leftarrow 1 \rightarrow E$ do:

 用当前策略 π_θ 采样轨迹 $\{s_1, a_1, r_1, s_2, a_2, r_2, \dots\}$

 为每一步数据计算 $\delta_t = r_t + \gamma V_w(s_{t+1}) - V_w(s_t)$

 更新价值参数 $w = w + \alpha_w \sum_t \delta_t \nabla_w V_w(s_t)$

 更新策略参数 $\theta = \theta + \alpha_\theta \sum_t \delta_t \nabla_\theta \log \pi_\theta(a_t | s_t)$

· end for

11. TRPO 算法

考虑在更新时找到一块信任区域(trust region), 在上面更新策略保证安全性.

策略目标: 考虑借助当前的 θ 找到更优参数 θ' , 使 $J(\theta') \geq J(\theta)$

将 $J(\theta)$ 写成 π_{θ} 的期望形式:

$$\begin{aligned} J(\theta) &= E_{s_0} [V^{\pi_{\theta}}(s_0)] \\ &= E_{\pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t V^{\pi_{\theta}}(s_t) - \sum_{t=1}^{\infty} \gamma^t V^{\pi_{\theta}}(s_t) \right] \\ &= -E_{\pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t (\gamma V^{\pi_{\theta}}(s_{t+1}) - V^{\pi_{\theta}}(s_t)) \right] \end{aligned}$$

新旧策略差距:

$$\begin{aligned} J(\theta') - J(\theta) &= E_{s_0} [V^{\pi_{\theta'}}(s_0)] - E_{s_0} [V^{\pi_{\theta}}(s_0)] \\ &= E_{\pi_{\theta'}} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] + E_{\pi_{\theta'}} \left[\sum_{t=0}^{\infty} \gamma^t (\gamma V^{\pi_{\theta}}(s_{t+1}) - V^{\pi_{\theta}}(s_t)) \right] \\ &= E_{\pi_{\theta'}} \left[\sum_{t=0}^{\infty} \gamma^t [r(s_t, a_t) + \gamma V^{\pi_{\theta}}(s_{t+1}) - V^{\pi_{\theta}}(s_t)] \right] \end{aligned}$$

时序差分定义为优势函数 A :

$$\begin{aligned} &= E_{\pi_{\theta'}} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_{\theta}}(s_t, a_t) \right] \\ &= \sum_{t=0}^{\infty} \gamma^t E_{s_t \sim p_t^{\pi_{\theta'}}} E_{a_t \sim \pi_{\theta'}(\cdot | s_t)} [A^{\pi_{\theta}}(s_t, a_t)] \\ &= \frac{1}{1-\gamma} E_{s \sim V^{\pi_{\theta'}}} E_{a \sim \pi_{\theta'}(\cdot | s)} [A^{\pi_{\theta}}(s, a)] \end{aligned}$$

最后一个等号用到状态访问分布的定义:

$$V^{\pi}(s) = (1-\gamma) \sum_{t=0}^{\infty} \gamma^t P_t^{\pi}(s)$$

所以只要我们能找到一个新策略, 使 $E_{s \sim V^{\pi_0}} [E_{a \sim \pi_{\theta'}(\cdot|s)} [A^{\pi_{\theta}}(s,a)]] \geq 0$
就能保证策略能单调递增, $J(\theta') \geq J(\theta)$

共轭梯度

核心思想: 直接计算 $x = H^{-1}g$, x 即参数更新方向. 假设满足 KL 距离约束的参数更新时的最大步长为 β , 有 $\frac{1}{2}(\beta x)^T H (\beta x) = \delta$.

$\Rightarrow \beta = \sqrt{\frac{2\delta}{x^T H x}}$, 此时更新方式为:

$$\theta_{k+1} = \theta_k + \sqrt{\frac{2\delta}{x^T H x}} x$$

具体流程:

初始化 $r_0 = g - Hx_0$, $p_0 = r_0$, $x_0 = 0$

for $k=0 \rightarrow N$ do:

$$\alpha_k = \frac{r_k^T r_k}{p_k^T H p_k}$$

$$x_{k+1} = x_k + \alpha_k p_k$$

如果 $r_{k+1}^T r_{k+1}$ 非常小, 则退出循环.

$$\beta_k = \frac{r_k^T r_{k+1}}{r_k^T r_k}$$

$$p_{k+1} = r_{k+1} + \beta_k p_k$$

end for

输出 x_{N+1}

广义优势估计

$$A_t^{(1)} = \delta_t = -V(s_t) + r_t + \gamma V(s_{t+1})$$

$$A_t^{(2)} = \delta_t + \gamma \delta_{t+1} = -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2})$$

$$A_t^{(3)} = \delta_t + \gamma \delta_{t+1} + \gamma^2 \delta_{t+2} = -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 V(s_{t+3})$$

⋮

$$A_t^{(K)} = \sum_{l=0}^{K-1} \gamma^l \delta_{t+l} = -V(s_t) + r_t + \gamma r_{t+1} + \dots + \gamma^{K-1} r_{t+K-1} + \gamma^K V(s_{t+K})$$

指数加权平均:

$$A_t^{\text{GAE}} = (1-\lambda)(A_t^{(1)} + \lambda A_t^{(2)} + \lambda^2 A_t^{(3)} + \dots)$$

$$= (1-\lambda)(\delta_t + \lambda(\delta_t + \gamma \delta_{t+1}) + \lambda^2(\delta_t + \gamma \delta_{t+1} + \gamma^2 \delta_{t+2}) + \dots)$$

$$= (1-\lambda)(\delta_t(1 + \lambda + \lambda^2 + \dots) + \gamma \delta_{t+1}(\lambda + \lambda^2 + \lambda^3 + \dots) + \gamma^2 \delta_{t+2}(\lambda^2 + \lambda^3 + \dots) + \dots)$$

$$= (1-\lambda) \left(\delta_t \frac{1}{1-\lambda} + \gamma \delta_{t+1} \frac{\lambda}{1-\lambda} + \gamma^2 \delta_{t+2} \frac{\lambda^2}{1-\lambda} + \dots \right)$$

$$= \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

其中, $\lambda \in [0, 1]$ 是 GAE 额外引入的超参数, 当 $\lambda = 0$ 时,

$$A_t^{\text{GAE}} = \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad \text{当 } \lambda = 1 \text{ 时,}$$

$$A_t^{\text{GAE}} = \sum_{l=0}^{\infty} \gamma^l \delta_{t+l} = \sum_{l=0}^{\infty} \gamma^l r_{t+l} - V(s_t)$$